

Criteria for the correctness of synchronization solutions.

1. Resources. A fundamental skill must be the ability to identify the resources for which processes compete. An example is an intersection: some students used to treat it as one resource, which of course does not allow vehicles to drive at the intersection at the same time. The issue of resource identification is so important that if you do it wrong, you can try to repair the program for half a year and it will still work poorly, because it has no chance of working in such a resource architecture. Of course, in the case of monitors, there must be as many monitors as there are separate resources. If resources are hierarchical, monitors should also be hierarchical.

2. Access to resources. You should ask yourself: how many semaphores should there be, how many conditional variables should there be? The situation with semaphores is a bit different, because they track access to critical sections, count available resources (for example, the number of free buffer elements), and finally sometimes serve as a place to stop the process (access scheduling). But in the case of monitors, the matter is exceptionally clear: there are as many condition variables as there are different Boolean conditions for the continuation of processes. Again: without correctly identifying the continuation conditions, the program cannot be run reasonably. I would like to point out that it is the errors in points 1 and 2 that cause active waiting so often in commercial programs, which makes the systems sluggish and inefficient. The best example: 2014 local elections in Poland

3. Multi-user resource. Using multi-user resources inside a monitor (or in a critical section protected by a semaphore) obviously does not allow for simultaneous access. Unfortunately, this is quite a common mistake. This error is facilitated by the synchronization style adopted by some programmers (unfortunately, also by teachers): the process performs the P operation on all semaphores associated with the resource, then uses the resource itself, and finally performs V for those semaphores that are associated with processes whose continuation conditions are fulfilled (a child game: I stand in a circle and choose who I prefer). In the case of monitors, this is the use of a single monitor within which the resource is used.

4. Active waiting. Inactive forms of waiting were developed to avoid repeated checking of the continuation conditions. This is another incarnation of a problem that existed in early computing: the processor actively waited for devices to be ready. Interrupts were invented that effectively solved the problem. At the level of access to resources, the equivalent is a semaphore/monitor. So active waiting cannot be accepted as an effective form of synchronization. Active waiting is unacceptable both in the internal form (the process loops in the monitor/class repeatedly checking the conditions) and in the external form (the process exits the monitor/class method without doing what it came there for, to try again, i.e. call the method again, maybe it will work this time).

5. The synchronizing operation is P/wait and the scheduling operation is V/signal. That is, the process refrains from running when it would be unsafe to do so and performs a P/wait synchronization operation. When a process has already used resources, which is related to changing the values of shared variables, it starts waiting processes. Of course, only those whose continuation condition has been met. Therefore, neither signal operations before using the resource nor wait operations after using it are

valid. Moreover, these last two errors are often related to active waiting: the process resumes all processes that can run in a loop until it reaches a situation when it can occupy the resource itself.

6. An obvious deadlock is when the process stops in its critical section.

7. The opposite of a deadlock, and equally incorrect, is examining the values of shared variables outside the critical section (and therefore also outside the monitor).

8. In the monitor, two situations can lead to the "stealing" of a continuation condition. The first one is the ladder of wait operation. Please look at the following code (of course the variable represents a boolean condition, it could be an expression on monitor variables)

1. if (!variable) c1.wait;
2. variable=false;
3. if (!othervar) c2.wait;
4. othervar=false;

This code is correct, although I disapprove of many of the conditions used in series. Well, maybe the conditions in a certain problem are hierarchical, which would justify the above construction. The absence of the *variable=false* statement means that the continuation condition may be stolen. Let us replace instructions 2 and 3, renumbering ABCD: 1-A, 2-C, 3-B, 4-D. Let us assume the following sequence of operations for the values *variable=true* and *othervar=false*:

process1 – A

process1 – B and stops on c2

process2 – A

process2 – B, even though it should stop because process1 would soon want to set the variable to false and block access.

Similarly, the ladder of signal operation can lead to condition stealing. Even the process itself can steal it! Let us assume the following situation in which process1 meets the continuation conditions for two processes, perhaps even completely separate conditions, for process2 and process3, waiting on different condition variables:

1. variable=true;
2. othervar=true;
3. c.signal;
4. c2.signal;

Let's assume the following sequence:

process1 – 1

process1 – 2

process1 – 3

process2 enters and consumes its condition by setting `variable=false`;

process3 enters and consumes its condition by setting `othervar=false`;

process3 enters and stays on the condition variable `c2`, because it has just consumed its continuation condition and now it is waiting for `othervar = true` to be set again, but no one has set it yet. Now process1 performs operation 4 and resumes process3, even though `othervar=false`!

9. Last, important thing: class. Of course, a monitor is a class, and in the case of semaphores, the class must include semaphore operations. Processes run outside the class, and call the class methods to access resources. So the class method should include taking the resource, using it and releasing it, and then resuming some waiting processes. If the resource is multi-user - it cannot be used internally in the monitor, we must develop separate operations for occupying and releasing the resource. We can safely use a multi-user resource in the class, but of course in a safe way. It is important that the class/monitor method does what it was called to do. So, if a method deals with the occupation of a resource, after its execution we must be sure that the resource is occupied. It is unacceptable that the method ends with setting some return parameter that informs that it should be called again. This is active waiting! In the correct solution, the designer does not even have to include processes that call the methods, the solution is a monitor/class itself (or a set of monitors/classes) that correctly implements access to shared resources.

I hope that these explanations will allow you to properly approach synchronization tasks, starting with the analysis of the problem, identification of resources and conditions of access to them. The rest is technical. The correct monitor method is as follows:

if (!can) c.wait;

can=true;

using the resource (if single-access);

someothercan=true;

c2.signal;

if others processes can run, then:

while (someothercan) c2.signal;

Of course, this last fragment is rare and may take other forms